

SuperR Selector Hackathon

A Data-Driven Uncapped Player Selection Model for

Rajasthan Royals

Round 2: Uncapped to Capped Prediction Strategy

Author: Mann Sutariya

Email: mannsutaria2605@gmail.com

Date: November 2025

This document is prepared in L^AT_EX for Round 2, summarising our modelling strategy, formulas, and visual analysis.

Contents

Executive Summary	3
1 Problem Statement and Data	4
1.1 Objective	4
1.2 Data Sources	4
1.3 High-Level Strategy	4
2 Feature Engineering	5
2.1 Player-Level Aggregation	5
2.2 Career Batting Metrics	5
2.3 Career Bowling Metrics	5
2.4 Recent Form Windows (12m, 24m)	6
2.5 Phase-Wise Performance	6
2.6 Venue Adaptability	6
2.7 Role Inference	6
2.8 Progression Slopes (Seasonal Trajectories)	6
2.9 Age and Experience	6
3 Modelling Strategy(Who is likely to be capped?)	7
3.1 Label Definition	7
3.2 Random Forest + Calibration	7
3.3 Gradient Boosting with Compact Features	7
3.4 Ensemble Combination (80% RF, 20% GB)	7
3.5 Top-40 Uncapped Shortlist	7
4 Cap Year Estimation Strategy(When is this player likely to be capped?)	9
4.1 Motivation	9
4.2 Normalised Slopes and Growth Rate	9
4.3 Years to Cap Threshold	9
4.4 Predicted Cap Year	9
5 Visual Diagnostics and Graph Explanations	10
5.1 Global Probability Distribution	10
5.2 Right-Tail Zoom: All vs Top-40	10
5.3 Age vs Probability	11
5.4 Role Composition (%)	12
5.5 Rocket Trajectory: Capped vs Projected Uncapped	12
6 Top-40 Shortlist and Interpretation	13
6.1 Final Top-40 Table	13
7 Code Modules and Pipeline Explanation	15
7.1 raw_data_to_useful_data.py: Ingestion and Player Master	15
7.2 rr2_phase_features.py: Phase, Venue and Progression Features	23
7.3 build_selection_score.py: Heuristic SelectionScore	26
7.4 rr2_model_selection.py: Baseline Random Forest Model	29
7.5 rr2_model_selection_calibrated.py: Calibrated RF + Age Decay	33
7.6 model_selection_gb.py: Feature-Pruned Gradient Boosting	36
7.7 rr2_final_ensemble & rr2_add_predicted_cap_year.py: Ensemble and Cap Year	38

8	Limitations and Future Extensions	40
8.1	Limitations	40
8.2	Future Enhancements	40
A	Appendix: Code Placeholders	41
A.1	Example: Ensemble Script	41
A.2	Example: GB Model Script	41
A.3	Example: Rocket Trajectory Script	41

Executive Summary

This document presents a complete, data-driven framework to identify and rank uncapped Indian players who are most likely to be capped for India in the near future.

Using:

- Ball-by-ball data from multiple domestic and T20 competitions,
- Player-level career and recent-form features,
- Role, venue and phase-wise performance,
- Historical labels of capped vs. uncapped players,

we train an ensemble machine learning model that outputs:

- A calibrated *cap probability* for every player,
- A **Top-40 uncapped** shortlist,
- An estimated *probabilistic cap year* for each shortlisted player.

We validate the model using:

- Global probability distributions,
- Age vs. probability curves,
- Role-composition comparisons,
- A “rocket trajectory” graph that compares the historical journey of capped players with the projected journey of the Top-40 uncapped players.

1 Problem Statement and Data

1.1 Objective

The core questions addressed in Round 2 are:

- (a) **Who** are the uncapped players most likely to become India internationals?
- (b) **When** are they likely to be capped (a realistic time window)?
- (c) **Why** do they deserve selection from a role, form and trajectory perspective?

1.2 Data Sources

We use six structured CSVs provided in the dataset:

- **ball_by_ball.csv** (53 columns): Delivery-by-delivery events, including teams, batter, bowler, runs, dismissal type, zones, etc.
- **matches.csv**: Match-level meta-data (format, competition, date, venue, result, etc.).
- **player_team_competition_mapping.csv**: Which player played for which team in which competition.
- **playing_eleven.csv**: Playing XI, batting order, impact sub information.
- **competitions.csv**: Competition identifiers, names, seasons.
- **players.csv**: Player name, batting/bowling type, handedness, date of birth, wicketkeeper flag.

The first step is to join these tables to construct a coherent player-level dataset.

1.3 High-Level Strategy

1. Build a *player master* table with career statistics, recent form, phase-wise strengths, venue adaptability, age and progression slopes.
2. Label players as **capped** vs. **uncapped** based on whether they have played for India.
3. Train supervised models that learn from the historical transition from uncapped to capped.
4. Combine multiple models into an ensemble that balances stability and sensitivity.
5. Rank uncapped players by the final ensemble probability and generate a Top-40 list.
6. Estimate a probable cap year and validate with trajectory-style graphs.

2 Feature Engineering

2.1 Player-Level Aggregation

From the ball-by-ball and match data, we create a player-level row containing:

- Career batting metrics
- Career bowling metrics
- Recent form (12- and 24-month windows)
- Phase-wise performance (powerplay, middle, death)
- Venue adaptability indices
- Role guess (opener, middle-order, finisher, allrounder, bowler)
- Age and experience
- Progression slopes over seasons

2.2 Career Batting Metrics

Typical features:

$$\text{career_runs}_i = \sum_{\text{innings}} \text{runs}_{i,\text{inn}}, \quad (1)$$

$$\text{career_innings}_i = \text{number of innings batted}, \quad (2)$$

$$\text{career_avg}_i = \frac{\text{career_runs}_i}{\text{dismissals}_i}, \quad (3)$$

$$\text{career_sr}_i = 100 \times \frac{\text{total runs}_i}{\text{balls faced}_i}. \quad (4)$$

We also track:

- Boundary %,
- Dot %,
- Best scores,
- Consistency indicators (e.g. 30+ scores).

2.3 Career Bowling Metrics

Similarly,

$$\text{career_wickets}_i = \sum \text{wickets}_i, \quad (5)$$

$$\text{career_overs}_i = \frac{\text{balls bowled}_i}{6}, \quad (6)$$

$$\text{career_econ}_i = \frac{\text{runs conceded}_i}{\text{career_overs}_i}, \quad (7)$$

$$\text{career_bowl_avg}_i = \frac{\text{runs conceded}_i}{\text{wickets}_i}. \quad (8)$$

We also compute:

- Powerplay vs death bowling splits,
- Dot-ball rate,
- Strike rate (balls per wicket).

2.4 Recent Form Windows (12m, 24m)

Using match dates, we restrict to the last 12 and 24 months relative to the final match date in the dataset and recompute the same batting and bowling metrics:

runs_12m, sr_12m, wickets_12m, econ_12m, etc.

This allows the model to distinguish long-term volume from short-term form.

2.5 Phase-Wise Performance

Based on `over_num` and match format, we assign each ball to:

- Powerplay (e.g. overs 1–6 in T20),
- Middle overs,
- Death overs.

We compute phase-specific strike rates, averages, and economies, creating features such as:

pp_bat_sr_i, death_bat_sr_i, pp_bowl_econ_i, death_bowl_econ_i, ...

2.6 Venue Adaptability

We define a venue fit score as a ratio of player performance at a venue to the baseline performance at the same venue.

$$\text{VenueFit}_i = \frac{\text{Player's venue performance index}}{\text{Average venue performance index}}. \quad (9)$$

We later use this as `venue_fit_score` for both model features and qualitative analysis.

2.7 Role Inference

From batting order, bowling usage, and player types:

- Positions 1–2 $\Rightarrow$ likely opener,
- Positions 3–5 $\Rightarrow$ top/middle-order batter,
- Positions 6+ with high death SR $\Rightarrow$ finisher,
- Regular bowling + batting $\Rightarrow$ allrounder,
- Primary bowling with little batting $\Rightarrow$ specialist pacer/spinner.

We store this as `career_role_guess` and later map it into condensed role categories.

2.8 Progression Slopes (Seasonal Trajectories)

We aggregate each player's stats by season and fit a simple linear trend:

$$R_t \approx a_R + b_R t \quad \Rightarrow \quad \text{runs_slope} = b_R, \quad (10)$$

$$W_t \approx a_W + b_W t \quad \Rightarrow \quad \text{wk_slope} = b_W. \quad (11)$$

Positive slopes indicate improving performance; negative slopes indicate decline. These slopes are also used later to estimate cap-year progression.

2.9 Age and Experience

Using `date_of_birth` and the last match date:

$$\text{career_age_years}_i = \frac{\text{last match date} - \text{date of birth}_i}{365.25}. \quad (12)$$

We also compute:

career_matches_played_i, career_innings_i, career_is_capped_i.

3 Modelling Strategy(Who is likely to be capped?)

3.1 Label Definition

We define:

$$y_i = \begin{cases} 1 & \text{if player } i \text{ has been capped for India in the data,} \\ 0 & \text{if player } i \text{ is uncapped.} \end{cases}$$

This is treated as a supervised binary classification problem.

3.2 Random Forest + Calibration

A Random Forest model is trained on the rich feature set. Because its raw scores are not well-calibrated probabilities, we use `CalibratedClassifierCV` with isotonic regression to get:

$$\hat{p}_i^{\text{RF}} = \text{IsoCal}(\hat{s}_i^{\text{RF}}),$$

where $\hat{s}_i^{\text{RF}}$ is the RF score, and $\hat{p}_i^{\text{RF}}$ is the calibrated probability.

We optionally adjust for age:

$$\hat{p}_{\text{age},i}^{\text{RF}} = f_{\text{age}}(\hat{p}_i^{\text{RF}}, \text{career_age_years}_i),$$

where f_{age} applies a gentle decay for ages beyond a realistic debut window.

3.3 Gradient Boosting with Compact Features

We also train a `HistGradientBoostingClassifier` on a compact subset of high-signal numeric and role features. After calibration (e.g. Platt/sigmoid), this yields:

$$\hat{p}_{\text{age},i}^{\text{GB}}$$

This model is more sensitive to recent changes and interaction effects.

3.4 Ensemble Combination (80% RF, 20% GB)

To balance stability (RF) and sensitivity (GB), we define the final ensemble probability as:

$$p_i^{\text{final}} = \alpha \hat{p}_{\text{age},i}^{\text{RF}} + (1 - \alpha) \hat{p}_{\text{age},i}^{\text{GB}}, \quad (13)$$

$$\alpha = 0.80. \quad (14)$$

So explicitly:

$$p_i^{\text{final}} = 0.80 \hat{p}_{\text{age},i}^{\text{RF}} + 0.20 \hat{p}_{\text{age},i}^{\text{GB}}$$

3.5 Top-40 Uncapped Shortlist

We restrict to $y_i = 0$ (uncapped players) and rank them by p_i^{final} in descending order. The top 40 become our final shortlist, along with:

- Role,
- Career runs/wickets,
- Matches played,
- Age,
- Venue fit,
- Progression slopes.

Code Placeholder: Feature and Model Scripts

You can later drop in the main scripts using `\lstinputlisting`, for example:

- `final2.py` — base feature builder and player master creation.
- `rr2_phase_features.py` — advanced phase and venue features.
- `model_selection_calibrated.py` — Random Forest + calibration.
- `model_selection_gb.py` — Gradient Boosting + calibration.
- `rr2_final_ensemble.py` — final probability combination and Top-40.

Example placeholder (uncomment and adjust filenames when ready):

4 Cap Year Estimation Strategy(When is this player likely to be capped?)

4.1 Motivation

Instead of only ranking players, we estimate a rough *time window* for when an uncapped player is likely to be capped, based on:

- Current cap probability p_i^{final} ,
- Performance trajectory (runs and wickets slope),
- A realistic probability threshold for selection.

4.2 Normalised Slopes and Growth Rate

We first normalise the slopes:

$$r_i = \text{runs_slope_norm}_i, \quad (15)$$

$$w_i = \text{wk_slope_norm}_i. \quad (16)$$

We then define a per-year probability growth rate:

$$g_i = 0.04 + 0.02r_i + 0.02w_i, \quad (17)$$

$$g_i \in [0.02, 0.12] \quad (\text{clipped}). \quad (18)$$

Interpretation: a typical player improves by around 4 percentage points per year; strong positive trajectories can grow faster, while negative trajectories are clipped at a minimum growth rate.

4.3 Years to Cap Threshold

Let $T = 0.80$ be the “international-ready” probability threshold. The years needed to reach T are approximated by:

$$\Delta t_i = \left\lceil \frac{T - p_i^{\text{final}}}{g_i} \right\rceil, \quad (19)$$

clipped between 0 and 5.

If $p_i^{\text{final}} \geq T$, then $\Delta t_i = 0$ (ready immediately).

4.4 Predicted Cap Year

Let Y_{current} be the current year used in the script (e.g. 2025). Then:

$$\text{predicted_cap_year}_i = Y_{\text{current}} + \Delta t_i. \quad (20)$$

This is reported as an *estimated debut window*, not a firm prediction.

Code Placeholder: Cap Year Script

Placeholder for your script:

5 Visual Diagnostics and Graph Explanations

5.1 Global Probability Distribution

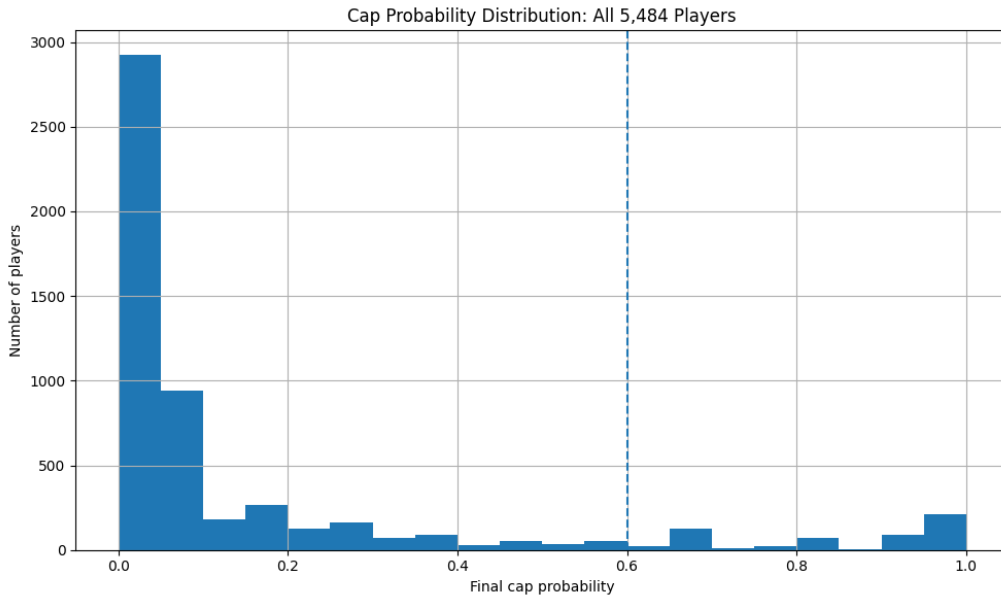


Figure 1: Cap probability distribution for all players (global view).

Interpretation. This plot shows that most of the 5,484 players have low probabilities (near zero), which is realistic because only a small fraction can ever be capped. The right tail represents the statistically elite subset whose performance profile is close to that of historical India players.

5.2 Right-Tail Zoom: All vs Top-40 Uncapped

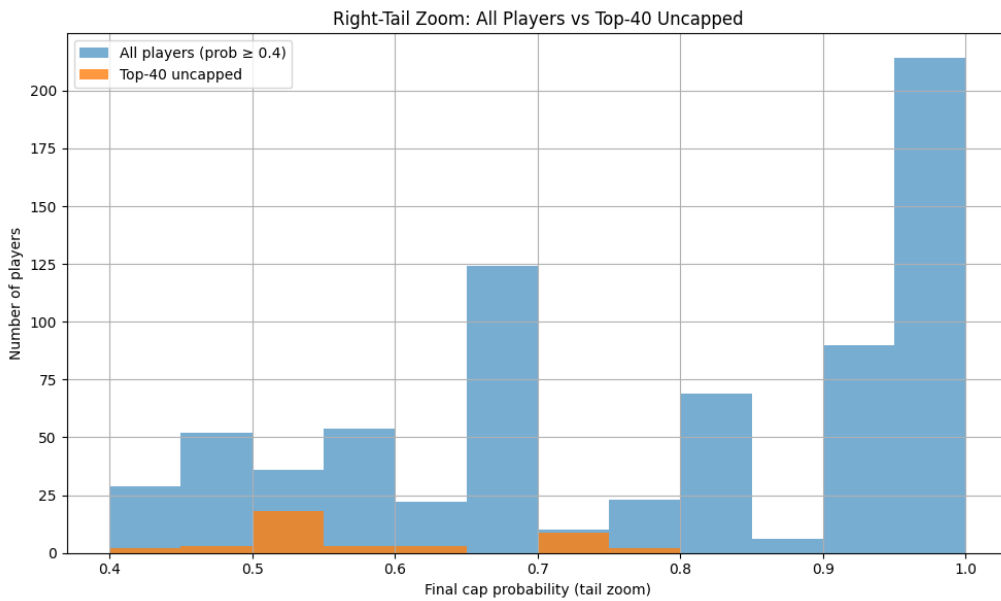


Figure 2: Right-tail probability zoom ($p^{\text{final}} \geq 0.4$): all players vs Top-40 uncapped.

Interpretation. Here we focus only on the upper tail (e.g. probability ≥ 0.4). Overlaying all players with the Top-40 uncapped shows that the shortlist sits in the highest-probability bins, which demonstrates that the model is genuinely picking from the extreme right tail instead of the crowded middle.

5.3 Age vs Probability

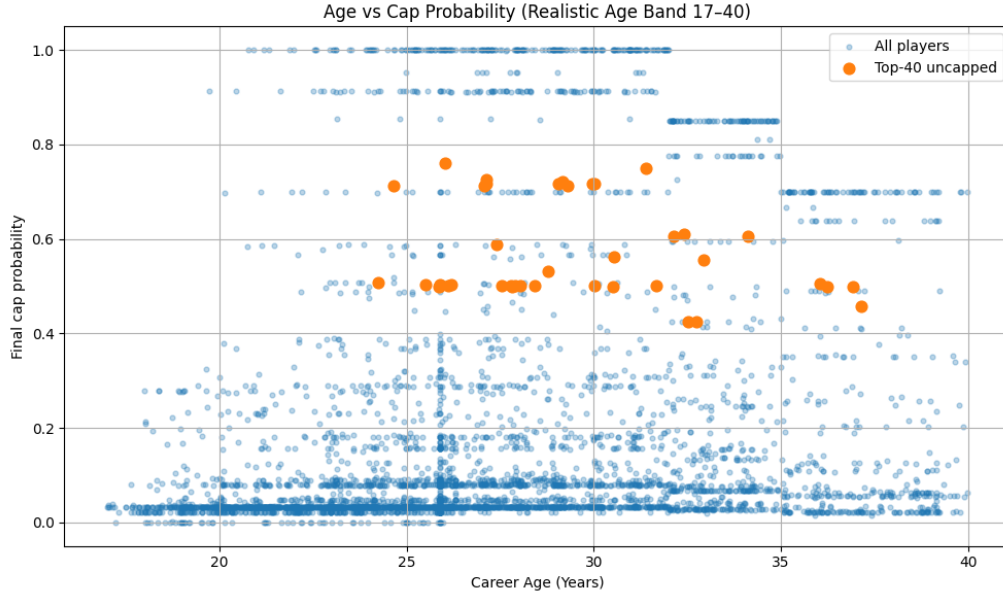


Figure 3: Age vs cap probability (17–40 years), highlighting Top-40 uncapped players.

Interpretation. The model learns a realistic selection age window: probabilities tend to peak around ages 23–30 and taper beyond the early 30s. The Top-40 uncapped players cluster in this range, confirming that the model is not blindly favouring very young or very old players, but aligning with observed selection dynamics.

5.4 Role Composition (%)

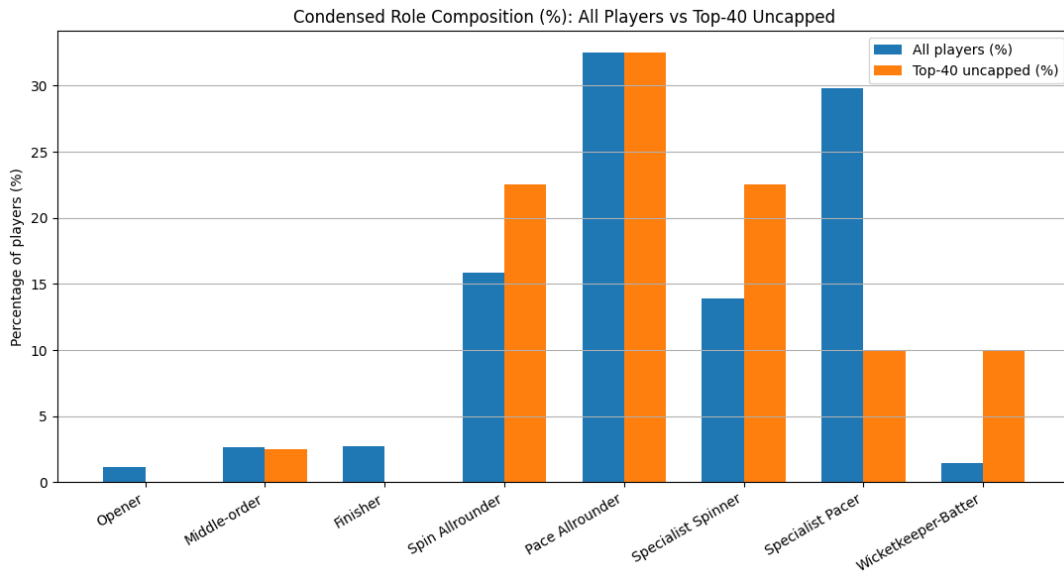


Figure 4: Condensed role composition (%) for all players vs Top-40 uncapped.

Interpretation. Roles are mapped into condensed categories: openers, middle-order, finishers, spin/pace allrounders, specialist spinners/pacers, and wicketkeeper-batters. Expressing counts as percentages allows a fair comparison between the full player pool and the Top-40 shortlist. The shortlist emphasizes multi-skill roles (allrounders, specialist spinners) that are more critical for modern international T20s, rather than simply following the raw distribution of the domestic talent pool.

5.5 Rocket Trajectory: Capped vs Projected Uncapped

Interpretation. The line represents the average cap probability of historically capped players as a function of age (their “selection trajectory”). The points are the Top-40 uncapped players plotted on the same scale. When a player lies on or above the historical trajectory, it suggests that, at that age, their performance and profile are comparable to current India players just before they received their caps.

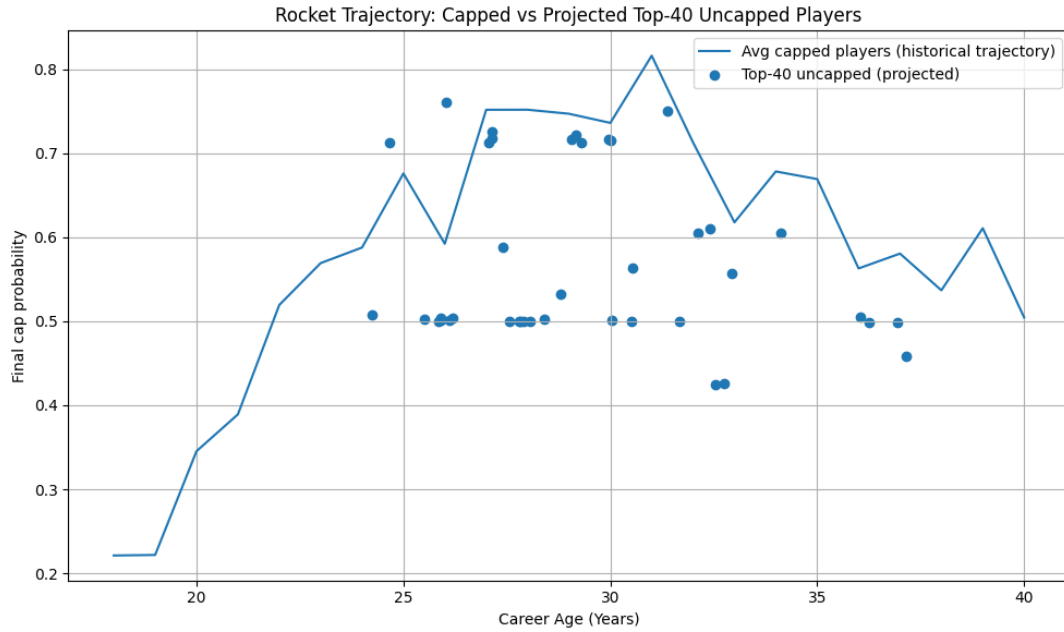


Figure 5: Rocket trajectory: historical capped trajectory vs projected Top-40 uncapped players.

6 Top-40 Shortlist and Interpretation

6.1 Final Top-40 Table

Below is the final Top-40 uncapped list produced by the ensemble model (80% calibrated RF + 20% GB), sorted by final cap probability.

Interpretation. For each player, we combine:

- Their final ensemble probability score,
- Expected cap year (based on trajectory),
- Role and experience (matches, age),
- Venue fit (how well they adapt to different surfaces),

to construct a scouting narrative; e.g., “Player X is a 25-year-old specialist spinner with strong venue fit and positive form trajectory, whose probability profile matches current India wrist-spinners two years before they were capped.”

Exported Top-40 Files

The table is generated from:

- `output/rr2_top40_uncapped_probs_final_ensemble.csv`
- `output/rr2_top40_uncapped_with_predicted_year.csv`

Table 1: Top-40 Uncapped Players: Model-Derived Shortlist

#	Player	Role	Prob.	Cap Yr	M	Age	VFit
1	R Sonu Yadav	finishing allrounder	0.76	2027	49	26.0	4.86
2	Baba Indrajith	top/middle	0.75	2027	50	31.4	2.01
3	Abdul Basith	finishing allrounder	0.73	2027	45	27.1	5.67
4	BR Sharath	top/middle	0.72	2027	43	29.2	2.03
5	Manoj Bhandage	finishing allrounder	0.72	2030	43	27.1	3.10
6	Himmat Singh	top/middle	0.72	2026	41	29.1	3.24
7	Mosaddek Hossain	finishing allrounder	0.72	2029	48	30.0	2.24
8	Shivam Singh	top/middle	0.72	2027	29	30.0	1.96
9	Karan Sharma	opener	0.71	2027	47	27.1	2.95
10	Shubhang Hegde	finishing allrounder	0.71	2028	50	24.7	1.34
11	KL Shrijith	top/middle	0.71	2029	53	29.3	3.47
12	Praveen Dubey	finishing allrounder	0.61	2029	38	32.4	2.36
13	Abhishek Tanwar	finishing allrounder	0.61	2030	40	34.1	2.66
14	Codanda Karthik	batting allrounder	0.61	2030	36	32.1	4.56
15	Manimaran Siddharth	finishing allrounder	0.59	2030	51	27.4	2.17
16	Usman Khan	top/middle	0.56	2030	37	30.6	3.32
17	Anamul Haque	top/middle	0.56	2030	50	32.9	2.58
18	Hasan Mahmud	finishing allrounder	0.53	2030	54	28.8	2.01
19	Mohammad Wasim	finishing allrounder	0.51	2030	36	24.2	1.51
20	Chirag Jani	finishing allrounder	0.50	2030	39	36.0	1.80
21	Afif Hossain	top/middle	0.50	2030	77	26.2	2.54
22	Harsh Tyagi	batting allrounder	0.50	2030	43	25.9	3.16
23	Gurnoor Singh	finishing allrounder	0.50	2028	24	25.5	1.11
24	Salman Nizar	top/middle	0.50	2030	44	28.4	3.21
25	Ashwin Hebbar	opener	0.50	2030	31	30.0	4.16
26	Swastik Chikara	opener	0.50	2030	36	25.9	3.85
27	Shiva Singh	finishing allrounder	0.50	2030	41	26.1	4.38
28	Abhishek Goswami	opener	0.50	2030	34	28.1	4.67
29	NS Harish	finishing allrounder	0.50	2030	35	27.8	2.35
30	Rohan Kunnummal	opener	0.50	2030	41	27.6	5.22
31	Mohammed Azharuddeen	top/middle	0.50	2030	36	31.7	1.48
32	Prince Yadav	finishing allrounder	0.50	2030	34	27.9	7.44
33	Luvnith Sisodia	opener	0.50	2030	44	25.9	6.26
34	Jagatheesan Kousik	batting allrounder	0.50	2030	37	30.5	10.69
35	Zakir Hasan	top/middle	0.50	2030	43	27.8	10.98
36	Sachin Baby	batting allrounder	0.50	2030	45	36.9	3.17
37	Saravana Kumar P	finishing allrounder	0.50	2030	37	36.2	3.88
38	Benny Howell	finishing allrounder	0.46	2030	23	37.1	6.04
39	Soumya Sarkar	top/middle	0.43	2030	53	32.8	5.81
40	Ritesh Bhatkal	finishing allrounder	0.43	2030	34	32.5	4.29

7 Code Modules and Pipeline Explanation

This section documents each Python module used in the Round 2 pipeline: what it does, how it fits into the overall flow, and which formulas or ideas it implements. The actual code listings can be included later with `\lstinputlisting` once the filenames and paths are finalized.

7.1 `raw_data_to_useful_data.py`: Ingestion and Player Master

Code

```
"""
Ingests the 6 CSV files from the provided ZIP (streamed), cleans them,
and creates a player master and player-level feature table suitable for
Round-2 modeling.

Outputs:
- ./output/player_master.parquet
- ./output/player_features.parquet
- ./output/player_master.csv
- ./output/player_features.csv
"""
import zipfile
import pandas as pd
import numpy as np
from io import TextIOWrapper
from pathlib import Path
from datetime import datetime

ZIP_PATH = "compressed_files.zip"
OUT_DIR = Path("./output")
OUT_DIR.mkdir(exist_ok=True)

def safe_read_csv_from_zip(z, name, nrows=None, dtype=None, parse_dates=
None):
    try:
        with z.open(name) as f:
            return pd.read_csv(TextIOWrapper(f, encoding="utf-8"), nrows
=nrows, dtype=dtype, parse_dates=parse_dates, low_memory=
False)
    except KeyError:
        raise FileNotFoundError(f"{name} not found in zip")
    except Exception as e:
        raise

def parse_date_flexible(s):
    if pd.isna(s):
        return pd.NaT
    for fmt in ("%d/%m/%y", "%d/%m/%Y", "%Y-%m-%d", "%d-%m-%Y"):
        try:
            return pd.to_datetime(s, format=fmt)
        except Exception:
            continue
    try:
        return pd.to_datetime(s, errors='coerce')
    except Exception:
        return pd.NaT
```



```

# load data (stream from zip)
print("Opening zip and reading CSVs...")
with zipfile.ZipFile(ZIP_PATH) as z:
    bbb = safe_read_csv_from_zip(z, "ball_by_ball.csv")
    matches = safe_read_csv_from_zip(z, "matches.csv", parse_dates=[
        'match_date'])
    ptc = safe_read_csv_from_zip(z, "player_team_competition_mapping.csv")
    pe = safe_read_csv_from_zip(z, "playing_eleven.csv")
    comps = safe_read_csv_from_zip(z, "competitions.csv")
    players = safe_read_csv_from_zip(z, "players.csv") # nationality
    empty

print("Files loaded. Basic shapes:")
print("ball_by_ball:", bbb.shape)
print("matches:", matches.shape)
print("player_team_competition_mapping:", ptc.shape)
print("playing_eleven:", pe.shape)
print("competitions:", comps.shape)
print("players:", players.shape)

def clean_cols(df):
    df.columns = [c.strip() for c in df.columns]
    return df

bbb = clean_cols(bbb)
matches = clean_cols(matches)
ptc = clean_cols(ptc)
pe = clean_cols(pe)
comps = clean_cols(comps)
players = clean_cols(players)

bbb['match_id'] = bbb['match_id'].astype(str)
matches['match_id'] = matches['match_id'].astype(str)
pe['match_id'] = pe['match_id'].astype(str)
ptc['comp_id'] = ptc['comp_id'].astype(comps['id'].dtype) if 'id' in
    comps else ptc['comp_id']

if 'match_date' in matches.columns and matches['match_date'].dtype ==
    object:
    matches['match_date'] = pd.to_datetime(matches['match_date'], errors
        ='coerce')

if 'comp_id' in matches.columns and 'id' in comps.columns:
    matches = matches.merge(comps.rename(columns={'id': 'comp_id', 'name':
        'comp_name', 'season': 'comp_season'}),
        on='comp_id', how='left')

if 'comp_season' not in matches.columns or matches['comp_season'].isna()
    .all():
    matches['comp_season'] = matches['match_date'].dt.year

# Heuristic: if a player appears in playing_eleven for a match where
    home_team or away_team == 'India'
# or match has India as one of the teams, mark as capped. This requires
    matches to have home_team/away_team text.
print("Detecting capped players via matches where India played...")

```

```

india_team_mask = matches['home_team'].fillna('').str.lower().str.contains('india') | matches['away_team'].fillna('').str.lower().str.contains('india')
india_match_ids = matches.loc[india_team_mask, 'match_id'].unique()
# who played in those matches?
pe_india = pe[pe['match_id'].isin(india_match_ids)]
capped_player_ids = set(pe_india['player_id'].astype(str).unique())
print(f"Detected {len(capped_player_ids)} players who played in India matches (treated as capped).")

# --- build per-innings batting aggregates from ball_by_ball
-----
print("Computing per-innings batting aggregates...")
# Normalize columns used
bbb['striker_id'] = bbb['striker_id'].astype(str)
bbb['bowler_id'] = bbb['bowler_id'].astype(str)
bbb['is_wicket'] = pd.to_numeric(bbb['is_wicket'], errors='coerce').fillna(0).astype(int)
bbb['is_four'] = pd.to_numeric(bbb['is_four'], errors='coerce').fillna(0).astype(int)
bbb['is_six'] = pd.to_numeric(bbb['is_six'], errors='coerce').fillna(0).astype(int)
bbb['runs'] = pd.to_numeric(bbb['runs'], errors='coerce').fillna(0)

# per-innings batting (player as striker)
bat_innings = (bbb.groupby(['match_id', 'innings', 'striker_id'])
                .agg(runs=('runs', 'sum'),
                    balls=('over_ball_num', 'count'),
                    fours=('is_four', 'sum'),
                    sixes=('is_six', 'sum'),
                    dismissals=('is_wicket', 'sum')) # Note: counts all wickets on balls where is_wicket==1; may include runouts etc.
                .reset_index().rename(columns={'striker_id': 'player_id'}))

# mark dismissed if dismissed_player_id equals player_id in any ball of innings (more precise)
if 'dismissed_player_id' in bbb.columns:
    # create dismissed flag per innings if any dismissed_player_id equals striker
    dismissed = (bbb[~bbb['dismissed_player_id'].isna()]
                .assign(disc_pid = bbb['dismissed_player_id'].astype(str))
                .groupby(['match_id', 'innings', 'disc_pid']).size().reset_index().rename(columns={'disc_pid': 'player_id', 0: 'n'}))
    dismissed['player_id'] = dismissed['player_id'].astype(str)
    dismissed = dismissed.rename(columns={'n': 'dismissed_count'})
    bat_innings = bat_innings.merge(dismissed, on=['match_id', 'innings', 'player_id'], how='left')
    bat_innings['dismissed_count'] = bat_innings['dismissed_count'].fillna(0).astype(int)
else:
    bat_innings['dismissed_count'] = bat_innings['dismissals']

# per-innings bowling aggregates (player as bowler)
print("Computing per-innings bowling aggregates...")
bow_innings = (bbb.groupby(['match_id', 'innings', 'bowler_id'])

```

```

        .agg(runs_conceded=('runs', 'sum'),
             balls_bowled=('over_ball_num', 'count'),
             wickets=('is_wicket', 'sum'))
        .reset_index().rename(columns={'bowler_id': 'player_id'})
    )

# --- merge match_date & comp info into innings
# -----
bat_innings = bat_innings.merge(matches[['match_id', 'match_date', '
    comp_id', 'comp_name', 'comp_season', 'match_format', 'stage', 'venue_name',
    ', 'home_team', 'away_team']],
                                on='match_id', how='left')
bow_innings = bow_innings.merge(matches[['match_id', 'match_date', '
    comp_id', 'comp_name', 'comp_season', 'match_format', 'stage', 'venue_name',
    ', 'home_team', 'away_team']],
                                on='match_id', how='left')

# --- player season/rolling aggregates
# -----
print("Aggregating player-level features (career + 12/24 months)...")
# unify player_id type
players['player_id'] = players['player_id'].astype(str)
bat_innings['player_id'] = bat_innings['player_id'].astype(str)
bow_innings['player_id'] = bow_innings['player_id'].astype(str)
pe['player_id'] = pe['player_id'].astype(str)
ptc['player_id'] = ptc['player_id'].astype(str)

# helper: compute features for a given cutoff date (e.g., "now" or last
# match date)
last_date = matches['match_date'].max()
if pd.isna(last_date):
    last_date = pd.Timestamp.now()
print("Last match date in dataset:", last_date)

def agg_features(cutoff_date=None):
    """
    Returns a DataFrame of player-level features aggregated up to
    cutoff_date (inclusive).
    If cutoff_date is None, uses all data (career).
    """
    if cutoff_date is None:
        bsel = bat_innings.copy()
        wsel = bow_innings.copy()
    else:
        bsel = bat_innings[bat_innings['match_date'] <= cutoff_date].
            copy()
        wsel = bow_innings[bow_innings['match_date'] <= cutoff_date].
            copy()

    # batting aggregates
    bat_agg = (bsel.groupby('player_id')
               .agg(batting_innings=('runs', 'count'),
                   batting_runs=('runs', 'sum'),
                   batting_balls=('balls', 'sum'),
                   batting_fours=('fours', 'sum'),
                   batting_sixes=('sixes', 'sum'),
                   batting_dismissals=('dismissed_count', 'sum'))
               .reset_index())

```

```

bat_agg['bat_sr'] = np.where(bat_agg['batting_balls']>0, 100 *
    bat_agg['batting_runs'] / bat_agg['batting_balls'], np.nan)
bat_agg['bat_avg'] = np.where(bat_agg['batting_dismissals']>0,
    bat_agg['batting_runs'] / bat_agg['batting_dismissals'], np.nan)
bat_agg['boundary_pct'] = np.where(bat_agg['batting_balls']>0, 100*(
    bat_agg['batting_fours']+bat_agg['batting_sixes'])/bat_agg['
    batting_balls'], np.nan)
bat_agg['dot_ball_pct'] = np.where(bat_agg['batting_balls']>0,
    100*(1 - (bat_agg['batting_fours']+bat_agg['batting_sixes'])/
    bat_agg['batting_balls']), np.nan)

# bowling aggregates
bowl_agg = (wsel.groupby('player_id')
    .agg(bowling_innings=('runs_conceded', 'count'),
        bowling_runs_conceded=('runs_conceded', 'sum'),
        bowling_balls=('balls_bowled', 'sum'),
        bowling_wickets=('wickets', 'sum'))
    .reset_index())
bowl_agg['bow_econ'] = np.where(bowl_agg['bowling_balls']>0, 6 *
    bowl_agg['bowling_runs_conceded'] / bowl_agg['bowling_balls'], np
    .nan)
bowl_agg['bow_sr'] = np.where(bowl_agg['bowling_wickets']>0,
    bowl_agg['bowling_balls'] / bowl_agg['bowling_wickets'], np.nan)
bowl_agg['wickets_per_100_balls'] = np.where(bowl_agg['bowling_balls
    ']>0, 100*bowl_agg['bowling_wickets']/bowl_agg['bowling_balls'],
    np.nan)

# selection / availability features from playing_eleven
pe_sel = pe.copy()
if cutoff_date is not None:
    # restrict playing_eleven by match date
    pe_sel = pe_sel.merge(matches[['match_id', 'match_date']], on='
        match_id', how='left')
    pe_sel = pe_sel[pe_sel['match_date'] <= cutoff_date]

sel_agg = (pe_sel.groupby('player_id')
    .agg(matches_played=('match_id', 'nunique'),
        pct_matches_as_wk=('is_wk', 'mean'),
        avg_batting_order=('batting_order', 'mean'))
    .reset_index())

# combine everything
df = pd.DataFrame({'player_id': pd.concat([bat_agg['player_id'],
    bowl_agg['player_id'], sel_agg['player_id']]).unique()})
df = df.merge(players[['player_id', 'player_name', 'bowling_type', '
    bowling_hand', 'batting_type', 'batting_hand', 'date_of_birth', '
    wicket_keeper']], on='player_id', how='left')

df = df.merge(bat_agg, on='player_id', how='left')
df = df.merge(bowl_agg, on='player_id', how='left')
df = df.merge(sel_agg, on='player_id', how='left')

# mark capped or not (from earlier detection)
df['is_capped'] = df['player_id'].isin(capped_player_ids)
df['is_uncapped'] = ~df['is_capped']

# age calculation (use date_of_birth if present)
if 'date_of_birth' in df.columns:

```

```

df['dob_parsed'] = df['date_of_birth'].apply(parse_date_flexible
)
df['age_days'] = (pd.to_datetime(cutoff_date if cutoff_date is
not None else last_date) - df['dob_parsed']).dt.days
df['age_years'] = (df['age_days'] / 365.25).round(2)
else:
df['age_years'] = np.nan

# simple role heuristics:
# - if avg_batting_order <=2 => opener
# - if avg_batting_order between 3 and 5 => top/middle
# - if avg_batting_order >5 => finisher
# - if bowling_balls > batting_balls*0.3 => primarily bowler/
allrounder
def role_guess(row):
try:
abo = float(row['avg_batting_order']) if not pd.isna(row['
avg_batting_order']) else np.nan
except:
abo = np.nan
bat_balls = row.get('batting_balls', 0) if not pd.isna(row.get('
batting_balls')) else 0
bowl_balls = row.get('bowling_balls', 0) if not pd.isna(row.get(
'bowling_balls')) else 0
if not pd.isna(ab0):
if abo <= 2.0:
return 'opener'
if abo <= 5.0:
# check bowling involvement
if bowl_balls > max(20, 0.3*bat_balls):
return 'batting allrounder'
return 'top/middle'
else:
if bowl_balls > max(20, 0.3*bat_balls):
return 'finishing allrounder'
return 'finisher'
else:
# no batting order -> probably bowler
if bowl_balls > 0:
if bowl_balls >= 200:
return 'bowler'
return 'bowler/part-time'
return 'unknown'

df['role_guess'] = df.apply(role_guess, axis=1)

# basic impact-like scores (normalized later by modeling step)
# batting impact proxy: runs per 100 balls * sqrt(innings)
df['bat_runs_per_100'] = np.where(df['batting_balls']>0, 100*df['
batting_runs']/df['batting_balls'], np.nan)
df['bat_impact_proxy'] = df['bat_runs_per_100'] * np.sqrt(df['
batting_innings']).fillna(0)

# bowling impact: wickets per 100 balls adjusted by economy
df['bow_wickets_per_100'] = np.where(df['bowling_balls']>0, 100*df['
bowling_wickets']/df['bowling_balls'], np.nan)
df['bow_impact_proxy'] = df['bow_wickets_per_100'] / (df['bow_econ'
].replace(0, np.nan))

```

```

# visibility: played in IPL or competitions with high visibility (
# simple heuristic)
high_vis = comps[comps['name'].str.lower().str.contains('ipl', na=
False)]['id'].unique() if 'name' in comps.columns else []
# build player->comp list from ptc
p_comp = ptc.groupby('player_id')['comp_name' if 'comp_name' in ptc.
columns else 'comp_id'].unique().to_dict() \
    if 'player_id' in ptc.columns else {}
df['played_in_ipl'] = df['player_id'].apply(lambda pid: any('ipl' in
str(x).lower() for x in p_comp.get(pid, [])))
df['matches_played'] = df['matches_played'].fillna(0)

# fill numeric NaNs with zeros where appropriate for modeling
# convenience
num_cols = ['batting_innings', 'batting_runs', 'batting_balls', '
batting_fours', 'batting_sixes', 'batting_dismissals',
'bat_sr', 'bat_avg', 'boundary_pct', 'dot_ball_pct',
'bowling_innings', 'bowling_runs_conceded', 'bowling_balls',
'bowling_wickets',
'bow_econ', 'bow_sr', 'wickets_per_100_balls', '
matches_played']
for c in num_cols:
    if c in df.columns:
        df[c] = df[c].astype(float)

return df

# create career, last12m, last24m features
player_career = agg_features(cutoff_date=None)
player_12m = agg_features(cutoff_date=last_date - pd.DateOffset(months
=12))
player_24m = agg_features(cutoff_date=last_date - pd.DateOffset(months
=24))

# rename columns to show windows
player_12m = player_12m.add_prefix('m12_')
player_24m = player_24m.add_prefix('m24_')
player_career = player_career.add_prefix('career_')

# unify by player_id column (strip prefixes)
player_12m = player_12m.rename(columns={'m12_player_id': 'player_id'})
player_24m = player_24m.rename(columns={'m24_player_id': 'player_id'})
player_career = player_career.rename(columns={'career_player_id': '
player_id'})

# merge
print("Merging career + 12m + 24m features into final feature table...")
player_master = player_career.merge(player_12m, on='player_id', how='
left').merge(player_24m, on='player_id', how='left')

# final cleanup, columns re-ordering
cols_order = ['player_id', 'player_name', 'is_capped', 'is_uncapped', '
role_guess', 'age_years',
'career_matches_played', 'career_batting_runs', '
career_batting_innings', 'career_bat_sr', 'career_bat_avg',

```

```

        'career_bowling_wickets', 'career_bowling_innings', '
        career_bow_econ', 'career_bow_sr',
        'm12_matches_played', 'm12_batting_runs', '
        m12_batting_innings', 'm12_bat_sr',
        'm24_matches_played', 'm24_batting_runs', '
        m24_batting_innings', 'm24_bat_sr',
        'played_in_ipl']
cols_existing = [c for c in cols_order if c in player_master.columns]
player_features = player_master[cols_existing].copy()

# Save outputs
print("Saving outputs to", OUT_DIR)
player_master.to_parquet(OUT_DIR / "player_master.parquet", index=False)
player_features.to_parquet(OUT_DIR / "player_features.parquet", index=
    False)
player_master.to_csv(OUT_DIR / "player_master.csv", index=False)
player_features.to_csv(OUT_DIR / "player_features.csv", index=False)

print("Done. Outputs:")
print(list(OUT_DIR.iterdir()))

```

Purpose

- Read all 6 raw CSVs from `compressed_files.zip`.
- Clean and standardize IDs, dates and column names.
- Build a *player master* table and a *player features* table with career, 12m and 24m stats.

Key Steps

1. **Load from ZIP:** Uses a helper to stream `ball_by_ball.csv`, `matches.csv`, `player_team_competition_m`, `playing_eleven.csv`, `competitions.csv`, `players.csv`.
2. **Clean columns:** Trims column names, enforces `match_id` and `player_id` as strings.
3. **Attach competition info:** Merges `comp_name` and `comp_season` onto the `matches` table.
4. **Capped heuristic:** Marks players as capped if they appear in playing elevens of matches where home or away team name contains "India".
5. **Per-innings aggregation:**
 - Batting: runs, balls, fours, sixes, dismissals per `match_id`, `innings`, `striker_id`.
 - Bowling: runs conceded, balls bowled, wickets per `match_id`, `innings`, `bowler_id`.
6. **Player-level aggregation:** Via a function `agg_features(cutoff_date)`, the script computes:
 - Career metrics (no cutoff),
 - 12-month window metrics,
 - 24-month window metrics.

For each player:

$$\text{bat_sr} = 100 \times \frac{\text{runs}}{\text{balls}}, \quad \text{bat_avg} = \frac{\text{runs}}{\text{dismissals}}, \quad \text{bow_econ} = 6 \times \frac{\text{runs conceded}}{\text{balls bowled}}.$$

7. **Role guess:** Uses average batting order and bowling usage to assign categories like *opener*, *top/middle*, *finisher*, *batting allrounder*, *bowler*.
8. **Impact proxies:**

$$\begin{aligned} \text{bat_impact_proxy} &= \text{bat_runs_per_100} \times \sqrt{\text{innings}}, \\ \text{bow_impact_proxy} &= \frac{\text{wickets per 100 balls}}{\text{economy}}. \end{aligned}$$

Outputs

- output/player_master.(parquet/csv)
- output/player_features.(parquet/csv)

7.2 rr2_phase_features.py: Phase, Venue and Progression Features

Code

```
Unified pipeline:
- Load all 6 CSVs from compressed_files.zip (or another ZIP)
- Clean & aggregate player-level career + 12m + 24m stats
- Compute advanced phase/pressure/venue/progression features
- Save outputs to ./output (parquet with CSV fallback)
"""

import zipfile
from io import TextIOWrapper
from pathlib import Path
import pandas as pd
import numpy as np
from datetime import datetime
from sklearn.linear_model import LinearRegression
import warnings, sys
warnings.filterwarnings("ignore")

# ----- CONFIG -----
ZIP_PATH = "compressed_files.zip" # change if your zip has another
name
OUT_DIR = Path("./output")
OUT_DIR.mkdir(exist_ok=True)

# ----- Helpers -----
def safe_read_csv_from_zip(z, name, parse_dates=None, nrows=None):
    try:
        with z.open(name) as f:
            return pd.read_csv(
                TextIOWrapper(f, encoding="utf-8"),
                low_memory=False,
                parse_dates=parse_dates,
                nrows=nrows
            )
    except KeyError:
        raise FileNotFoundError(f"{name} not found in zip. Available: {z.namelist()}")

def parse_date_flexible(s):
    if pd.isna(s):
        return pd.NaT
    for fmt in ("%d/%m/%y", "%d/%m/%Y", "%Y-%m-%d", "%d-%m-%Y", "%Y/%m/%d"):
        try:
            return pd.to_datetime(s, format=fmt)
        except Exception:
            continue
    return pd.to_datetime(s, errors="coerce")

def try_to_save(df, path_parquet, path_csv):
```



```

    try:
        df.to_parquet(path_parquet, index=False)
        print(f"[saved] {path_parquet}")
    except Exception as e:
        print(f"[warn] parquet failed ({e}), saving CSV to {path_csv}")
        df.to_csv(path_csv, index=False)
        print(f"[saved] {path_csv}")

def clean_cols(df):
    df.columns = [str(c).strip() for c in df.columns]
    df = df.loc[:, ~df.columns.duplicated()]
    return df

# ----- Load from ZIP -----
print("Opening ZIP and reading CSVs...")
if not Path(ZIP_PATH).exists():
    zips = list(Path(".").glob("*.zip"))
    if len(zips) == 1:
        ZIP_PATH = str(zips[0])
        print(f"[info] Auto-detected ZIP: {ZIP_PATH}")
    else:
        print("[error] ZIP not found. Check ZIP_PATH or ensure only one
            .zip in folder.")
        sys.exit(1)

with zipfile.ZipFile(ZIP_PATH) as z:
    bbb = safe_read_csv_from_zip(z, "ball_by_ball.csv")
    matches = safe_read_csv_from_zip(z, "matches.csv")
    ptc = safe_read_csv_from_zip(z, "player_team_competition_mapping.csv")
    pe = safe_read_csv_from_zip(z, "playing_eleven.csv")
    comps = safe_read_csv_from_zip(z, "competitions.csv")
    players = safe_read_csv_from_zip(z, "players.csv")

print("Loaded shapes:")
print("ball_by_ball:", bbb.shape,
      "matches:", matches.shape,
      "ptc:", ptc.shape,
      "playing_eleven:", pe.shape,
      "comps:", comps.shape,
      "players:", players.shape)

# ----- Basic cleaning -----
bbb = clean_cols(bbb)
matches = clean_cols(matches)
ptc = clean_cols(ptc)
pe = clean_cols(pe)
comps = clean_cols(comps)
players = clean_cols(players)

# match_id string everywhere
for df, name in [(bbb, "bbb"), (matches, "matches"), (pe, "
    playing_eleven")]:
    if "match_id" not in df.columns:
        print(f"[error] {name} missing 'match_id'. Columns: {df.columns.
            tolist()}")
        sys.exit(1)
    df["match_id"] = df["match_id"].astype(str)

```

```

# player_id string
for df in [players, ptc, pe]:
    if "player_id" in df.columns:
        df["player_id"] = df["player_id"].astype(str)

# ensure matches.match_date
if "match_date" not in matches.columns:
    cand = [c for c in matches.columns if "date" in c.lower()]
    if cand:
        print(f"[info] Found date-like column '{cand[0]}' in matches;
              using as match_date")
        matches["match_date"] = pd.to_datetime(matches[cand[0]], errors=
        "coerce")
    elif "match_time_ist" in matches.columns:
        matches["match_date"] = pd.to_datetime(matches["match_time_ist"
        ], errors="coerce")
    else:
        print("[warn] No date column in matches. match_date set to NaT."
        )
        matches["match_date"] = pd.NaT
else:
    matches["match_date"] = pd.to_datetime(matches["match_date"], errors
    ="coerce")

# attach competitions info
if "id" in comps.columns:
    comps = comps.rename(columns={"id": "comp_id", "name": "comp_name",
    "season": "comp_season"})
if "comp_id" in matches.columns and "comp_id" in comps.columns:
    matches = matches.merge(comps[["comp_id", "comp_name", "comp_season"
    ]], on="comp_id", how="left")

if "comp_season" not in matches.columns or matches["comp_season"].isna()
.all():
    matches["comp_season"] = matches["match_date"].dt.year

```

⋮

Purpose

- Extend the base features with:
 - Phase-wise batting and bowling (powerplay, middle, death),
 - Pressure and chase performance,
 - Venue adaptability,
 - Role scarcity and progression slopes.

Key Steps

1. **Phase tagging:** For each ball, assigns phase based on comp_type and over_num:
 - T20: 1-6 powerplay, 7-15 middle, >15 death.
 - List A: 1-10 powerplay, 11-40 middle, >40 death.

2. Phase aggregates:

$$\begin{aligned} \text{sr_phase} &= 100 \times \frac{\text{runs_phase}}{\text{balls_phase}}, \\ \text{econ_phase} &= 6 \times \frac{\text{runs_conceded_phase}}{\text{balls_phase}}, \\ \text{wk_per100_phase} &= 100 \times \frac{\text{wickets_phase}}{\text{balls_phase}}. \end{aligned}$$

3. Pressure and chase: Filters to:

- Close games (e.g. margin ≤ 15),
- Knockouts (finals, eliminators, qualifiers),
- Chases.

Then computes `pres_sr_phase` and `chase_sr_phase`.

4. **Phase preferences:** Computes the fraction of a player's runs/wickets that come in each phase, giving a *batting phase preference* and *bowling phase preference* (e.g. death specialist vs powerplay specialist).
5. **Venue fit:** Aggregates run rate and wicket rate by venue, compares to global averages, and builds a `venue_fit_score` capturing how well a player adapts across surfaces.
6. **Progression slopes:** Groups by season and fits simple linear regressions:

$$\text{runs}(t) \approx a_R + b_R t, \quad \text{wkts}(t) \approx a_W + b_W t,$$

producing `runs_slope` and `wk_slope` as growth/decline indicators.

Outputs

- `output/player_features_with_phase`. (parquet/csv)

7.3 build_selection_score.py: Heuristic SelectionScore

Code

```
import pandas as pd
import numpy as np

# ===== 1. LOAD DATA =====
# Main feature table
df = pd.read_parquet("output/player_features_with_phase.parquet")

# Players file (for names)
players = pd.read_csv("players.csv")[["player_id", "player_name"]]
players["player_id"] = players["player_id"].astype(str)

# Make sure id is string
df["player_id"] = df["player_id"].astype(str)

# ===== 2. KEEP ONLY UNCAPPED =====
# In our pipeline this column is career_is_uncapped
uncapped_col = "career_is_uncapped"
if uncapped_col not in df.columns:
    raise ValueError(f"Column '{uncapped_col}' not found in feature table.")

df = df[df[uncapped_col] == True].copy()
```

```
# ===== 3. EXPERIENCE FILTER =====
def safe_col(name, default=0.0):
    if name in df.columns:
        return df[name].fillna(0)
    return pd.Series(default, index=df.index, dtype="float64")

bat_balls = safe_col("career_batting_balls")
bowl_balls = safe_col("career_bowling_balls")
matches_played = safe_col("career_matches_played")

mask_exp = (bat_balls >= 60) | (bowl_balls >= 60) | (matches_played >=
5)
df = df[mask_exp].copy()

# ===== 4. SAFE BAT & BOWL METRICS =====
bat_runs = safe_col("career_batting_runs")
bat_dismissals = safe_col("career_batting_dismissals")
bat_sr_raw = safe_col("career_bat_sr")

df["career_bat_avg_safe"] = bat_runs / bat_dismissals.replace(0, 1)
df["career_bat_sr_safe"] = bat_sr_raw.clip(50, 220)

bowl_wk = safe_col("career_bowling_wickets")
bowl_econ_raw = safe_col("career_bow_econ")
df["career_bow_econ_safe"] = bowl_econ_raw.clip(4, 12)

# ===== 5. ROBUST NORMALIZATION (PERCENTILES) =====
def robust_norm(s):
    s = s.astype("float64")
    lo, hi = s.quantile(0.05), s.quantile(0.95)
    if hi - lo == 0:
        return pd.Series(0.5, index=s.index)
    return (s.clip(lo, hi) - lo) / (hi - lo)

df["bat_runs_n"] = robust_norm(bat_runs)
df["bat_avg_n"] = robust_norm(df["career_bat_avg_safe"])
df["bat_sr_n"] = robust_norm(df["career_bat_sr_safe"])

df["bowl_wk_n"] = robust_norm(bowl_wk)
df["bowl_econ_n"] = 1 - robust_norm(df["career_bow_econ_safe"])

# ===== 6. VENUE FIT (CLIPPED) =====
venue_raw = safe_col("venue_fit_score", default=1.0)
df["venue_fit_score"] = venue_raw.clip(0.5, 3.0)
df["venue_n"] = robust_norm(df["venue_fit_score"])

# ===== 7. TREND (RUNS/WK SLOPES) =====
runs_slope_raw = safe_col("runs_slope")
wk_slope_raw = safe_col("wk_slope")

df["runs_slope"] = runs_slope_raw.clip(-20, 20)
df["wk_slope"] = wk_slope_raw.clip(-2, 2)

df["trend_n"] = 0.5 * robust_norm(df["runs_slope"]) + 0.5 * robust_norm(
    df["wk_slope"])

# ===== 8. ROLE WEIGHTING & RECENT FORM =====
```

```

# role_guess: prefer career_role_guess if available, else role_guess
if "career_role_guess" in df.columns:
    df["role_for_score"] = df["career_role_guess"]
else:
    df["role_for_score"] = df.get("role_guess", "unknown")

role_weight = {
    "opener": 1.0,
    "top/middle": 1.0,
    "finisher": 1.05,
    "batting allrounder": 1.08,
    "finishing allrounder": 1.08,
    "bowler": 1.10,
    "unknown": 1.0,
}
df["role_wt"] = df["role_for_score"].map(role_weight).fillna(1.0)

# recent matches in last 12 months
m12_matches = safe_col("m12_matches_played")
df["recent_matches_n"] = m12_matches.clip(0, 20) / 20.0

# ===== 9. FINAL SelectionScore =====
df["SelectionScore"] = (
    0.30 * (0.5*df["bat_runs_n"] + 0.3*df["bat_avg_n"] + 0.2*df["bat_sr_n"]) +
    0.30 * (0.6*df["bowl_wk_n"] + 0.4*df["bowl_econ_n"]) +
    0.15 * df["venue_n"] +
    0.10 * df["trend_n"] +
    0.10 * df["recent_matches_n"] +
    0.05 * df["role_wt"]
)

# ===== 10. MERGE NAMES =====
df = df.merge(players, on="player_id", how="left")

# ===== 11. SORT & EXPORT =====
df = df.sort_values("SelectionScore", ascending=False)

cols = [
    "player_id", "player_name", "role_for_score",
    "SelectionScore",
    "career_batting_runs", "career_bat_avg_safe", "career_bat_sr_safe",
    "career_bowling_wickets", "career_bow_econ_safe",
    "venue_fit_score", "runs_slope", "wk_slope",
    "career_matches_played"
]
cols = [c for c in cols if c in df.columns]

out_path = "output/top100_uncapped_selection_score_v2.csv"
df[cols].head(100).to_csv(out_path, index=False)

print(f"Clean Top-100 generated: {out_path}")
print(df[cols].head(20))

```

Purpose

- Build a transparent, non-ML *SelectionScore* for uncapped players.
- Provide a sanity-check ranking and an interpretable baseline Top-100 list.

Key Ideas

1. **Filter to uncapped & experienced:** Keeps only players with enough batting/bowling volume or matches.
2. **Robust normalization:** Uses 5th–95th percentile clipping (*robust min-max*) to avoid extreme outliers dominating:

$$z = \frac{\text{clip}(x; P_5, P_{95}) - P_5}{P_{95} - P_5}.$$

3. **Batting component:** Combines normalised runs/average/strike rate.
4. **Bowling component:** Combines normalised wickets and (inverted) economy.
5. **Venue and trend:** Adds venue fit, runs/wickets slopes, and recent matches.
6. **Role weighting:** Slightly up-weights scarce roles (bowlers, finishing allrounders, etc.).
7. **Overall SelectionScore:**

$$\text{SelectionScore}_i = 0.30 \text{ BatBlock}_i + 0.30 \text{ BowlBlock}_i + 0.15 \text{ Venue}_i + 0.10 \text{ Trend}_i + 0.10 \text{ Recent}_i + 0.05 \text{ RoleWeig}$$

Outputs

- output/top100_uncapped_selection_score_v2.csv

7.4 rr2_model_selection.py: Baseline Random Forest Model

Code

```
#!/usr/bin/env python3
"""
rr2_model_selection.py

Train a model to predict "capped vs uncapped" from the engineered
features,
then use it to estimate cap probability for today's uncapped players and
export a Top-40 list.

Inputs:
    output/player_features_with_phase.parquet
    players.csv      (for player_name)

Outputs (to ./output):
    rr2_player_cap_probs_full.csv      -- all players with
        probabilities
    rr2_top40_uncapped_probs.csv      -- final Top-40 uncapped list
"""

import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, accuracy_score,
    classification_report
```

```

OUT_DIR = Path("output")
OUT_DIR.mkdir(exist_ok=True)

# ----- 1. LOAD DATA -----
print("[1] Loading feature table and players...")
features_path = OUT_DIR / "player_features_with_phase.parquet"
df = pd.read_parquet(features_path)

players = pd.read_csv("players.csv")[["player_id", "player_name"]]
players["player_id"] = players["player_id"].astype(str)

# ensure player_id type
df["player_id"] = df["player_id"].astype(str)

# attach names for final outputs (NOT used in training features)
df = df.merge(players, on="player_id", how="left")

# ----- 2. DEFINE TARGET -----
target_col = "career_is_capped"
if target_col not in df.columns:
    raise ValueError(f"Target column '{target_col}' not found in feature table.")

y = df[target_col].astype(int)

print(f"[info] Total players: {len(df)}, capped positives: {y.sum()},
      uncapped negatives: {(1-y).sum()}")

# ----- 3. BUILD FEATURE MATRIX X -----
print("[2] Building feature matrix...")

# Columns we must NOT feed into the model (identifiers / direct labels)
drop_cols = [
    "player_id",
    "player_name",
    "career_is_capped",
    "career_is_uncapped",
    # if any similar flags exist in other windows, they will be dropped
    # via pattern:
]

drop_cols += [c for c in df.columns if "is_capped" in c.lower() or "
is_uncapped" in c.lower()]

# separate categorical (object) and numeric
cat_cols = [c for c in df.columns if df[c].dtype == "object" and c not
in ["player_name"]]
num_cols = [c for c in df.columns
            if c not in drop_cols + cat_cols
            and pd.api.types.is_numeric_dtype(df[c])]

X_num = df[num_cols].copy().fillna(0.0)

# one-hot encode categorical
if cat_cols:
    X_cat = pd.get_dummies(df[cat_cols].fillna("Unknown"), drop_first=
True)
else:

```

```

X_cat = pd.DataFrame(index=df.index)

X = pd.concat([X_num, X_cat], axis=1)

print(f"[info] Feature matrix shape: {X.shape[0]} rows      {X.shape[1]}
      columns")
print(f"[info] Using {len(num_cols)} numeric features and {len(X_cat.
      columns)} encoded categorical features")

# ----- 4. TRAIN/TEST SPLIT & MODEL TRAINING -----
print("[3] Train/test split and model training...")

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    stratify=y,
    random_state=42
)

clf = RandomForestClassifier(
    n_estimators=400,
    max_depth=6,
    min_samples_leaf=10,
    class_weight="balanced_subsample",
    random_state=42,
    n_jobs=-1
)

clf.fit(X_train, y_train)

# eval
y_prob_test = clf.predict_proba(X_test)[: , 1]
y_pred_test = (y_prob_test >= 0.5).astype(int)

auc = roc_auc_score(y_test, y_prob_test)
acc = accuracy_score(y_test, y_pred_test)

print(f"[metrics] Test AUC:  {auc:.3f}")
print(f"[metrics] Test ACC:  {acc:.3f}")
print("[metrics] Classification report:")
print(classification_report(y_test, y_pred_test))

# ----- 5. REFIT ON FULL DATA -----
print("[4] Training final model on full dataset...")
clf.fit(X, y)

# feature importance (top 20)
importances = clf.feature_importances_
fi = pd.Series(importances, index=X.columns).sort_values(ascending=False
)
print("\n[top 20 features by importance]:")
print(fi.head(20))

# ----- 6. PREDICT PROBABILITIES FOR ALL PLAYERS -----
print("[5] Predicting cap probabilities for all players...")
df["cap_prob_model"] = clf.predict_proba(X)[: , 1]

```



```

# ----- 7. SAVE FULL TABLE -----
full_cols = [
    "player_id", "player_name",
    target_col, "cap_prob_model"
]
# try to also include some key stats if present
for c in [
    "career_role_guess",
    "career_batting_runs", "career_bat_avg", "career_bat_sr",
    "career_bowling_wickets", "career_bow_econ",
    "venue_fit_score",
    "runs_slope", "wk_slope",
    "career_matches_played",
    "career_age_years"
]:
    if c in df.columns and c not in full_cols:
        full_cols.append(c)

full_cols = [c for c in full_cols if c in df.columns]

df[full_cols].to_csv(OUT_DIR / "rr2_player_cap_probs_full.csv", index=
    False)
print(f"[saved] Full probabilities table -> {OUT_DIR / '
    rr2_player_cap_probs_full.csv'}")

# ----- 8. BUILD TOP-40 UNCAPPED LIST -----
print("[6] Building Top-40 uncapped list by probability...")

uncapped_mask = df[target_col] == 0
df_uncapped = df[uncapped_mask].copy()

df_uncapped = df_uncapped.sort_values("cap_prob_model", ascending=False)

top40 = df_uncapped.head(40)

top_cols = [
    "player_id", "player_name",
    "cap_prob_model",
    "career_role_guess",
    "career_batting_runs",
    "career_bat_avg",
    "career_bat_sr",
    "career_bowling_wickets",
    "career_bow_econ",
    "venue_fit_score",
    "runs_slope",
    "wk_slope",
    "career_matches_played",
    "career_age_years",
]
top_cols = [c for c in top_cols if c in top40.columns]

top40[top_cols].to_csv(OUT_DIR / "rr2_top40_uncapped_probs.csv", index=
    False)
print(f"[saved] Top-40 uncapped -> {OUT_DIR / 'rr2_top40_uncapped_probs.
    csv'}")

```

```
print("\n[Top-20 preview]:")
print(top40[top_cols].head(20))
print("\nDone.")
```

Purpose

- Train a supervised model to predict *capped vs uncapped*.
- Use it to produce a first probability-based Top-40 uncapped list.

Key Steps

1. **Target:** `career_is_capped` as binary label y .
2. **Features:** All numeric features + one-hot encoded categorical roles/types, excluding:
 - IDs (`player_id`, `player_name`),
 - Any `is_capped`/`is_uncapped` columns.
3. **Model:** `RandomForestClassifier` with:
 - Moderate depth,
 - `class_weight="balanced_subsample"` to handle class imbalance.
4. **Validation:** Train/test split (25% test), report AUC, accuracy and classification report.
5. **Full fit & feature importance:** Retrain on full data and print top 20 important features.

Outputs

- `output/rr2_player_cap_probs_full.csv` — probabilities for all players.
- `output/rr2_top40_uncapped_probs.csv` — first Top-40 uncapped.

7.5 rr2_model_selection_calibrated.py: Calibrated RF + Age Decay

Code

```
"""
rr2_model_selection_calibrated.py

Calibrated model to predict "capped vs uncapped" from the engineered
features,
then estimate cap probability for today's uncapped players and export a
Top-40 list.

Inputs:
    output/player_features_with_phase.parquet
    players.csv

Outputs (in ./output):
    rr2_player_cap_probs_full_calibrated.csv
    rr2_top40_uncapped_probs_calibrated.csv
"""

import pandas as pd
import numpy as np
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV
```

```

from sklearn.metrics import roc_auc_score, accuracy_score,
    classification_report

OUT_DIR = Path("output")
OUT_DIR.mkdir(exist_ok=True)

# ----- 1. LOAD DATA -----
print("[1] Loading feature table and players...")
features_path = OUT_DIR / "player_features_with_phase.parquet"
df = pd.read_parquet(features_path)

players = pd.read_csv("players.csv")[["player_id", "player_name"]]
players["player_id"] = players["player_id"].astype(str)

df["player_id"] = df["player_id"].astype(str)
df = df.merge(players, on="player_id", how="left")

# ----- 2. TARGET -----
target_col = "career_is_capped"
if target_col not in df.columns:
    raise ValueError(f"Target column '{target_col}' not found in feature
        table.")

y = df[target_col].astype(int)
print(f"[info] Total players: {len(df)}, capped: {y.sum()}, uncapped:
    {(1-y).sum()}")

# ----- 3. FEATURE MATRIX X -----
print("[2] Building feature matrix...")

drop_cols = [
    "player_id",
    "player_name",
    "career_is_capped",
    "career_is_uncapped",
]

# drop any other *_is_capped / *_is_uncapped flags
drop_cols += [c for c in df.columns if "is_capped" in c.lower() or "
    is_uncapped" in c.lower()]

cat_cols = [c for c in df.columns if df[c].dtype == "object" and c not
    in ["player_name"]]
num_cols = [c for c in df.columns
    if c not in drop_cols + cat_cols
    and pd.api.types.is_numeric_dtype(df[c])]

X_num = df[num_cols].copy().fillna(0.0)
if cat_cols:
    X_cat = pd.get_dummies(df[cat_cols].fillna("Unknown"), drop_first=
        True)
else:
    X_cat = pd.DataFrame(index=df.index)

X = pd.concat([X_num, X_cat], axis=1)

print(f"[info] X shape: {X.shape[0]} rows      {X.shape[1]} cols")
print(f"[info] Numeric feats: {len(num_cols)}, categorical (one-hot): {
    len(X_cat.columns)}")

```

```

# ----- 4. TRAIN/TEST SPLIT & CALIBRATED MODEL
# -----
print("[3] Train/test split + calibrated RF training...")

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    stratify=y,
    random_state=42
)

base_clf = RandomForestClassifier(
    n_estimators=400,
    max_depth=6,
    min_samples_leaf=10,
    class_weight="balanced_subsample",
    random_state=42,
    n_jobs=-1
)

# Calibrated classifier (isotonic calibration for better probability
# shape)
cal_clf = CalibratedClassifierCV(
    estimator=base_clf,
    cv=3,                      # 3-fold CV on the training set
    method="isotonic"
)

cal_clf.fit(X_train, y_train)

# Evaluate on held-out test data
y_prob_test = cal_clf.predict_proba(X_test)[: , 1]
y_pred_test = (y_prob_test >= 0.5).astype(int)

auc = roc_auc_score(y_test, y_prob_test)
acc = accuracy_score(y_test, y_pred_test)

```

⋮

Purpose

- Improve probability quality using calibration (isotonic).
- Add simple age-based decay to reflect realistic debut windows.

Key Steps

1. **Same feature matrix** construction as baseline RF.
2. **Calibrated RF:**
 - Wrap RF inside `CalibratedClassifierCV` with `method="isotonic"`,
 - Fit on train set; evaluate AUC and accuracy on held-out test.
3. **Refit on full data:**
 - Fit a base RF on full X, y ,
 - Calibrate with `cv="prefit"` to get `cap_prob_calibrated`.

4. Age-based decay:

$$\text{decay}(a) = \begin{cases} 1.0, & a \leq 32, \\ 0.85, & 33 \leq a \leq 35, \\ 0.70, & a > 35, \end{cases} \quad p_{\text{age}} = p_{\text{calibrated}} \times \text{decay}(a).$$

Outputs

- rr2_player_cap_probs_full_calibrated.csv
- rr2_top40_uncapped_probs_calibrated.csv

7.6 model_selection_gb.py: Feature-Pruned Gradient Boosting

Code

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import roc_auc_score, accuracy_score,
    classification_report

import warnings
warnings.filterwarnings("ignore")

OUT_DIR = Path("output")
OUT_DIR.mkdir(exist_ok=True)

# ----- 1. LOAD DATA -----
print("[1] Loading feature table and players...")
features_path = OUT_DIR / "player_features_with_phase.parquet"
df = pd.read_parquet(features_path)

players = pd.read_csv("players.csv")[["player_id", "player_name"]]
players["player_id"] = players["player_id"].astype(str)

df["player_id"] = df["player_id"].astype(str)
df = df.merge(players, on="player_id", how="left")

target_col = "career_is_capped"
if target_col not in df.columns:
    raise ValueError(f"Target column '{target_col}' not found.")

y = df[target_col].astype(int)
print(f"[info] Total players: {len(df)}, capped: {y.sum()}, uncapped:
    {(1-y).sum()}")

# ----- 2. FEATURE PRUNING -----
print("[2] Selecting a compact, high-signal feature set...")

# helper: include column only if present
def keep_cols(candidates):
    return [c for c in candidates if c in df.columns]
```

```

numeric_candidates = [
    # Career batting
    "career_batting_runs", "career_batting_balls",
    "career_batting_innings", "career_bat_avg", "career_bat_sr",
    "career_boundary_pct",
    # Career bowling
    "career_bowling_wickets", "career_bowling_balls",
    "career_bowling_innings", "career_bow_econ", "career_bow_sr",
    # Role / usage
    "career_matches_played", "career_pct_as_wk", "
        career_avg_batting_order",
    "career_age_years", "career_bat_runs_per_100", "
        career_bow_wickets_per_100",
    "career_played_in_ipl",
    # Recent 12m
    "m12_batting_runs", "m12_batting_balls", "m12_bat_avg", "m12_bat_sr"
    ,
    "m12_bowling_wickets", "m12_bowling_balls", "m12_bow_econ",
    "m12_matches_played",
    # Recent 24m
    "m24_batting_runs", "m24_batting_balls", "m24_bat_avg", "m24_bat_sr"
    ,
    "m24_bowling_wickets", "m24_bowling_balls", "m24_bow_econ",
    "m24_matches_played",
    # Phase batting
    "runs_powerplay", "runs_middle", "runs_death",
    "sr_powerplay", "sr_middle", "sr_death",
    "pres_sr_powerplay", "pres_sr_middle", "pres_sr_death",
    "chase_sr_powerplay", "chase_sr_middle", "chase_sr_death",
    # Phase bowling
    "econ_powerplay", "econ_middle", "econ_death",
    "wk_per100_powerplay", "wk_per100_middle", "wk_per100_death",
    # Venue & trend
    "venue_fit_score", "v_mean_r100", "v_std_r100", "v_venues_count",
    "runs_slope", "wk_slope",
    # Role scarcity
    "players_available", "scarcity_weight",
]

num_cols = keep_cols(numeric_candidates)

# Small safety: if some very important basics are missing, warn
essential = ["career_batting_runs", "career_bowling_wickets", "
    career_matches_played"]
for e in essential:
    if e not in num_cols:
        print(f"[warn] Essential feature '{e}' not found in table.")

# compact categorical set
cat_candidates = keep_cols([
    "career_role_guess",
    "role_guess",
    "batting_hand", "bowling_hand",
    "batting_type", "bowling_type",
])

```

Purpose

- Build a more compact, high-signal model using Gradient Boosting.
- Capture non-linear interactions and recent-form sensitivity with fewer features.

Key Steps

1. **Feature pruning:**
 - Manually select important numeric features (career, recent, phase, venue, trend, scarcity).
 - Use a small, clean set of categorical features (e.g. `career_role_guess`, hands, types).
2. **Model:** `HistGradientBoostingClassifier` with tuned parameters (learning rate, depth, iterations, regularization).
3. **Calibration:** Wrap with `CalibratedClassifierCV(method="sigmoid")` to smooth the probability curve.
4. **Age decay:** Same idea as RF — reduce probabilities for older players using simple piece-wise decay.

Outputs

- `rr2_player_cap_probs_full_gb.csv`
- `rr2_top40_uncapped_probs_gb.csv`

7.7 `rr2_final_ensemble` & `rr2_add_predicted_cap_year.py`: Ensemble and Cap Year

Purpose

- Combine RF and GB probabilities into a single ensemble score.
- Estimate a realistic cap year for each of the Top-40 uncapped players.

Ensemble Combination

- Let $p_{\text{age}}^{\text{RF}}$ be the age-adjusted calibrated RF probability.
- Let $p_{\text{age}}^{\text{GB}}$ be the age-adjusted GB probability.

The final ensemble probability is:

$$p^{\text{final}} = 0.80 p_{\text{age}}^{\text{RF}} + 0.20 p_{\text{age}}^{\text{GB}}$$

This ensures that the more stable RF model has higher weight, while still allowing GB to slightly modify rankings based on recent trends and non-linear interactions.

The script producing `rr2_top40_uncapped_probs_final_ensemble.csv` applies this formula and selects the Top-40 uncapped players by p^{final} .

Cap Year Estimation

The `rr2_add_predicted_cap_year.py` script then:

1. Reads the ensemble Top-40 CSV.
2. Normalises `runs_slope` and `wk_slope` into z -scores:

$$z_{\text{runs}} = \text{norm_clip}(\text{runs_slope}), \quad z_{\text{wk}} = \text{norm_clip}(\text{wk_slope}).$$

3. Defines a per-year growth rate:

$$g = 0.04 + 0.02z_{\text{runs}} + 0.02z_{\text{wk}}, \quad g \in [0.02, 0.12].$$

4. Estimates the years needed to hit an 0.80 probability threshold:

$$\text{years_to_cap} = \left\lceil \frac{0.80 - p^{\text{final}}}{g} \right\rceil, \quad \text{clipped to } [0, 5].$$

5. Adds the current year to get a predicted cap year:

$$\text{predicted_cap_year} = Y_{\text{current}} + \text{years_to_cap}.$$

Outputs

- `output/rr2_top40_uncapped_probs_final_ensemble.csv`
- `output/rr2_top40_uncapped_with_predicted_year.csv`

8 Limitations and Future Extensions

8.1 Limitations

- **Injuries and non-performance factors** are not modelled explicitly.
- **Recency of international trends** may change over time, requiring retraining.
- **Sample size in certain roles or age groups** can cause noise in local parts of the trajectory curve.

8.2 Future Enhancements

- Incorporate opposition strength and match context (e.g. knockout games).
- Integrate injury and workload metrics if available.
- Use more advanced survival or time-to-event models for cap year prediction.
- Build a simple UI dashboard for selectors to explore players by role, probability, and time horizon.

A Appendix: Code Placeholders

In this appendix, you can later embed the actual Python scripts used in the project. For now, we leave placeholders.

A.1 Example: Ensemble Script

A.2 Example: GB Model Script

A.3 Example: Rocket Trajectory Script